OblivRec: Final Presentation

CS670 FINAL PROJECT · 30-MINUTE RECORDED PRESENTATION · 3 SPEAKERS

How to use this. Each slide has a target time in brackets, and that time includes letting a figure or the demo sit on screen, not only speech. *On screen* is what the deck shows. The paragraph under it is what the speaker says. Stage directions are in italics. A **[TRANSITION]** line marks each handoff.

The narration is about 3,310 words, which is roughly 24 minutes of speech at 140 words a minute. The slide targets add up to about 28 minutes, and the difference is time spent on figures and the demo. That leaves a margin under the 30-minute limit. If a rehearsal runs long, cut dwell time before cutting words.

Every number in this script is checked against the committed data, and the table at the end says where each one comes from. If a benchmark is ever re-run, re-check that table before recording.

Speakers and timing

ARC	SPEAKER	TOPIC	SLIDES	SPOKEN	TARGET WITH FIGURES
1	Shravan Agrawal	Two halves of a private recommender	1 to 8	~8:00	8:40
2	Shrasti Dwivedi	Why the fetch has to be private	9 to 16	~6:50	9:00
3	Mainak Sarkar	What it costs, and where it breaks	17 to 26	~8:50	10:35
			Total	**~23:40**	**28:15**

The speakers follow the attribution in the mid-term report the team submitted, so the two submissions tell the same story. The team can reassign, but the course handout allows a viva at any stage, so whoever presents an arc should be able to answer the questions listed at its end.

The deck

The slides are `OblivRec_final_presentation.pptx`: 26 numbered slides for the talk, then five backup slides (27 to 31) for questions, which are not presented. Each slide's speaker notes hold its narration from this file, word for word, so the notes and this script cannot disagree. The deck embeds its fonts (Manrope and IBM Plex Mono), so it looks the same on any machine running PowerPoint. A PDF export sits beside it as a fallback.

Figures on screen

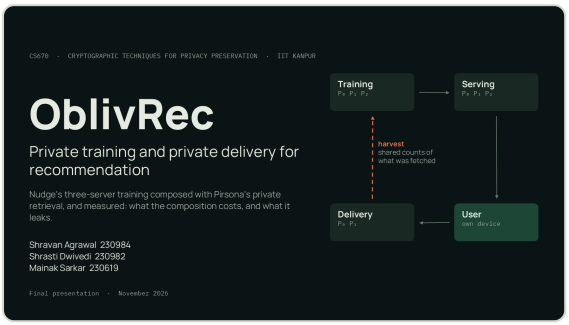
All figures come from `mingw32-make figures` and live in `report/figures/`. Where a slide shows one panel of a figure, the panel is cut from that same file and not redrawn.

LABEL	FILE	SLIDE
F1	`fig_attack.pdf`	10 (both panels), 11 (left panel)
F2	`fig_keysize.pdf`	backup 27
F3	`fig_pir_cost.pdf`	backup 30
F4	`fig_ell.pdf`	backup 28

LABEL	FILE	SLIDE
F5	`fig_distinguisher.pdf`	backup 29
F6	`fig_decoy.pdf`	12
F7	`fig_sweep.pdf`	20 (right panel)
F8	`fig_compose.pdf`	18 (left panel), 22 (right panel)
F9	`fig_stages.pdf`	19 (left panel)
F10	`fig_dp.pdf`	14 (left panel), 15 (right panel)
F11	`fig_malicious.pdf`	23
F12	`fig_timing.pdf`	24

ARC 1 · Two halves of a private recommender

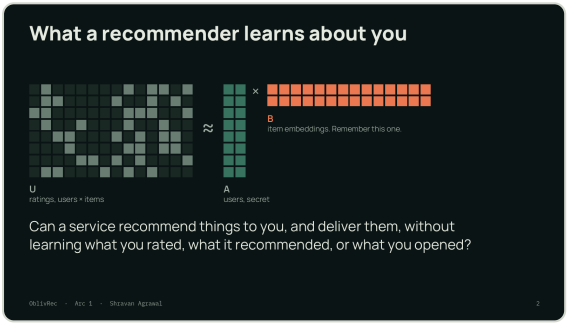
Shravan Agrawal · target 8:40



01 OblivRec 0:20

On screen: title, the three names, CS670, IIT Kanpur.

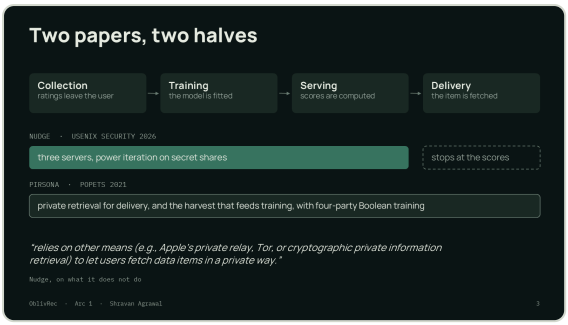
Hello. We are Shravan Agrawal, Shrasti Dwivedi and Mainak Sarkar, and this is OblivRec, our CS670 project on privacy-preserving recommendation. In the next thirty minutes we will show you what we built, what it costs, and where it breaks.



02 What a recommender learns about you 1:00

On screen: a ratings matrix, users as rows and items as columns, factorised into two thin matrices labelled A and B.

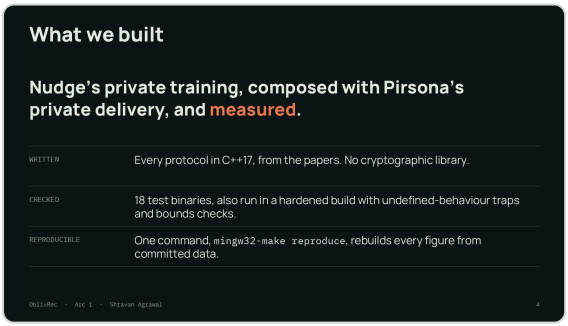
Every recommender works the same way. You tell it what you liked, and it learns what you will like next. Collaborative filtering does this by factorising a large matrix of ratings, one row per user and one column per item, into two small matrices. One describes users and one describes items. The item matrix is called B, and it will matter a great deal later. The catch is that the service has to see your ratings to do any of this. Your ratings are your preferences, and preferences reveal far more than people expect. So the question this project asks is simple to state. Can a service recommend things to you, and deliver them, without ever learning what you rated, what it recommended, or what you actually opened?



03 Two papers, two halves 1:30

On screen: the four stages of a recommender (collection, training, serving, delivery), with the stages each paper covers marked beneath. Nudge (USENIX Security 2026) covers the first three and stops at the scores. Pirsona (PoPETs 2021) covers delivery with four-party training. Below, Nudge's own sentence: *"relies on other means ... to let users fetch data items in a private way."*

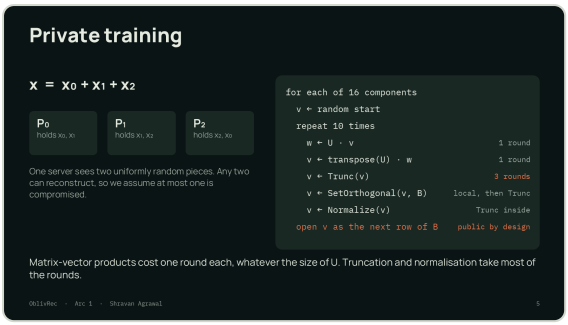
Our instructor pointed us at two papers, and each one solves half of this problem. The first is Nudge, from USENIX Security 2026. It trains the recommender on secret-shared ratings across three servers, at the scale of the Netflix dataset. But Nudge stops at the scores. In its own words, it relies on other means to let users fetch data items in a private way. So once you know which film to watch, fetching it tells the server exactly which film that was. The second paper is Pirsona, from PoPETs 2021. It does the fetching privately, using private information retrieval, and it has a lovely trick where the servers collect the next round of training data straight out of those private fetches. But its training runs a four-party protocol, and Nudge's training needs one fewer server that has to be trusted not to collude. So neither paper does both halves well. Nudge has the training we want and Pirsona has the private delivery, and nobody had put them together, because nobody had built both.



04 What we built 0:45

On screen: one line. "Nudge's private training + Pirsona's private delivery, composed, and measured."

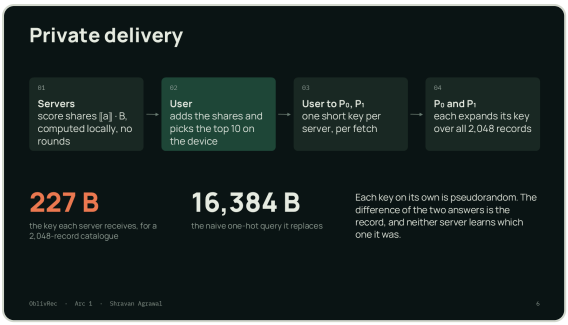
That is what we did. OblivRec composes Nudge's private training with Pirsona's private delivery into one system, and then measures the composition, because a composition is only interesting if you know what it costs. Everything we show you today runs. We wrote the cryptographic core ourselves, from the original papers rather than from a library. And every figure on these slides is regenerated from committed data by a single command.



05 Private training 1:30

On screen: a user's ratings split into three pieces, one per server. Below, the power iteration loop, with the two matrix-vector products marked "one round each, whatever the matrix size" and truncation and normalisation marked "most of the rounds".

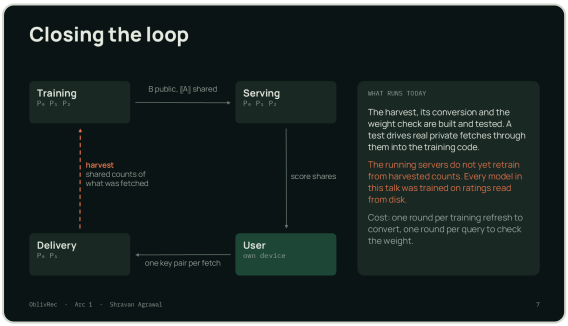
Here is how the training half works. Each user's ratings are split into three random-looking pieces, one per server, so that any two servers together could recover them but any single server sees only noise. This is two-out-of-three replicated secret sharing. Now, factorising a matrix under secret sharing is normally expensive, because multiplying two secret values forces the servers to talk to each other. Nudge's insight is to factorise with power iteration, which is mostly matrix-times-vector work. Under this kind of sharing, a matrix-vector product is cheap. Each server multiplies its own pieces locally, and only the result is exchanged, in a single round, however large the matrix is. Most of the rounds go to two other operations: truncation, which keeps fixed-point numbers from overflowing, and normalisation. There is one design choice to hold onto. As each part of the item matrix B converges, it is opened in the clear. B is public, by design. That is what makes training affordable, and it is also the reason delivery has to be private, as Shrasti will show you.



06 Private delivery 1:30

On screen: scores computed from a secret vector and the public B, the user taking the top ten on their own device, then two servers each receiving a short key.

Now the delivery half. Once B is public, scoring is almost free. Your score for every item is your own secret vector times the public matrix B, and multiplying a secret-shared vector by a public matrix needs no communication at all. Each server computes its share of your scores alone. The servers never assemble a ranking. They hand you your score shares, you add them up on your own device, and you pick your top ten locally. Then you have to fetch those ten items, and this is where Pirsona's half comes in. We use two-server private information retrieval, built on a distributed point function. You send each server a short key, 227 bytes for our catalogue. Each key on its own looks random. The server runs it across every item in the catalogue and returns a combination, and when you put the two answers together you get exactly the record you asked for. Neither server learns which one. We wrote this distributed point function ourselves, and Mainak will show you how we checked it.



07 Closing the loop 1:20

On screen: a circle. Private fetch, then the harvested shares, then the next round of private training, then back to delivery.

There is one more piece, and it is what makes this a system rather than two halves bolted together. When a server answers a private fetch, it has already computed a secret share of a vector with a one at the item you fetched. Pirsona's trick is to keep that share instead of throwing it away. Summed over all your fetches, those shares become a secret-shared record of what you consumed, which is exactly the input the next round of training needs. So the servers collect the next round's training data straight out of this round's private fetches, with no separate upload. Turning those shares into training input costs one round per training refresh. Delivery feeds training, and training feeds delivery. To be precise about how far this goes: the harvest is built and tested, and a test drives real private fetches through it into the training code. Our running servers do not yet retrain from it, and every model we show was trained on ratings read from disk.



08 It runs end to end

0:45

On screen: the terminal output of

```
demo --a model/out/A_private_fss_b128_ell10.bin \
     --b model/out/B_private_fss_b128_ell10.bin \
     --expect
model/out/top10_u42_private_fss_b128_ell10.txt
```

showing ten titles, Toy Story first, and "All 10 records fetched privately and verified byte-exact".

And it does run, end to end. This is our demo serving a model that was trained privately, using the version of training that reveals nothing at all. It returns ten real film titles for user forty-two, Toy Story first, and every one of them was fetched by private information retrieval and checked byte for byte against the catalogue. The ranking also matches an independently computed reference on all ten positions. Nothing on this screen was ever visible to a server. But there is a question we have not answered yet. Does delivery really need to be private at all?

[TRANSITION] Shrasti.

Questions a viva will ask about Arc 1

Why is scoring free? B is public, and a public matrix applied to a secret-shared vector is a linear operation, so each server computes its share of the scores locally with no rounds at all. Evidence: `src/serve/serve.cpp`, and `tests/test_serve.cpp`, where all 1,682 scores match the reference exactly.

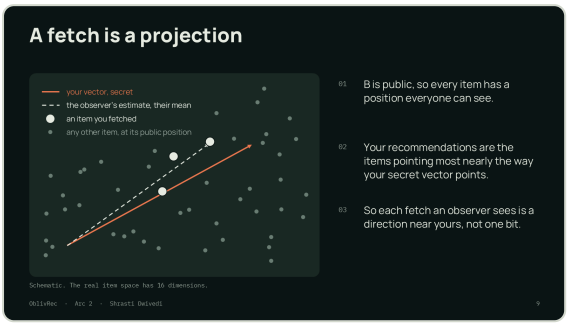
Why does the user pick the top ten, and not the servers? The user reconstructs the scores and selects locally, so no server ever learns the selection. An oblivious top-k protocol was designed and then cut on 6 September, because it would protect something that is already protected. Evidence: the Decisions Log in `design/ARCHITECTURE-draft-v1.md`.

How do you know a server cannot tell which item was fetched? Each key on its own is pseudorandom, and the server evaluates every index unconditionally. We also measured it. Over 1,800 recorded queries every query frame is 228 bytes and every reply is 265 bytes, whatever the index, and a classifier trained on the raw wire bytes scores between 0.490 and 0.500 against a chance line of 0.5. Evidence: `bench/results/distinguisher_result.jsonl`, figure F5.

What if two servers collude? Then the shares reconstruct and there is no protection. That is the assumption the whole system rests on, and we state it as such.

ARC 2 · Why the fetch has to be private

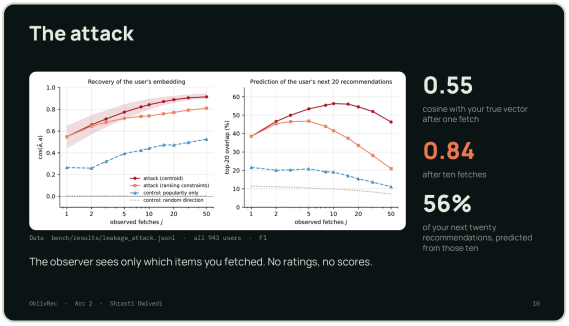
Shrasti Dwivedi · target 9:00



09 A fetch is a projection 1:00

On screen: points in a space labelled "items (public)", a hidden arrow labelled "your vector", and three highlighted fetched items near its direction.

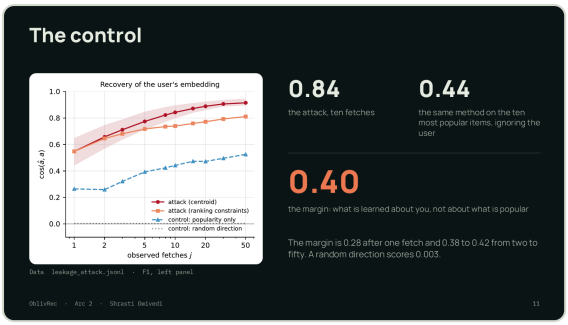
Thank you, Shravan. Let me start with the design choice he asked you to hold onto. B is public. Every item has a public position in a sixteen-dimensional space, and your recommendations are simply the items that point most nearly in the same direction as your own secret vector. So think about what an observer learns if they see which items you fetch, and nothing else. Each fetch is not one bit of information. It is a direction in a space everyone can see, and it tells the observer that your vector points somewhere near it. Collect a few of those, and the observer can start to triangulate your vector. That is the attack, and we measured it.



10 The attack 1:30

On screen: **F1.** Left, cosine similarity against the number of observed fetches. Right, the share of the user's next twenty recommendations predicted.

Here is the result, across all 943 users of MovieLens. The attack is deliberately simple. The observer averages the public vectors of the items you fetched and uses that as a guess of your secret vector. After a single fetch, the guess already has a cosine similarity of 0.55 with your true vector. After ten fetches it reaches 0.84. And the number that matters most to a non-specialist is on the right. From your first ten fetches alone, the observer predicts 56 percent of your next twenty recommendations, things you have not fetched yet. So without private delivery, a server that never sees a single rating still learns your taste well enough to predict what you will watch next. Neither of the papers we started from measures this, because neither one implements both halves.

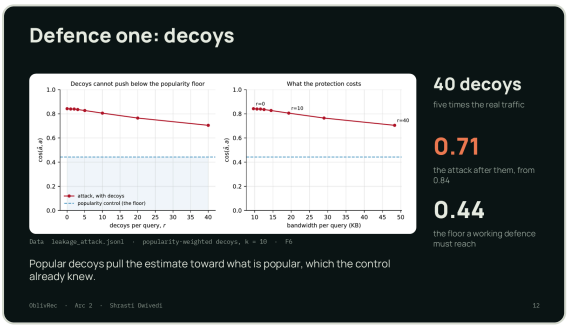


11 The control

1:15

On screen: the left panel of **F1** enlarged, beside the three numbers that matter: the attack at 0.84, the popularity control at 0.44, and the margin between them.

The part of this experiment we care about most is the control. Popular films sit near the top of almost everyone's list. So a guessing method can look impressive while having learned nothing about you personally, only what is popular, which is public anyway. So we ran the identical method on the ten most popular films in the catalogue, ignoring the user completely. It reaches a cosine of 0.44. The attack reaches 0.84. The gap between them, about 0.40, is the part that is genuinely about you, and it stays at roughly that size for every number of fetches from two to fifty. We also report a random guess, which sits at zero, as it should. We would rather show you the control than let a large number speak for itself.

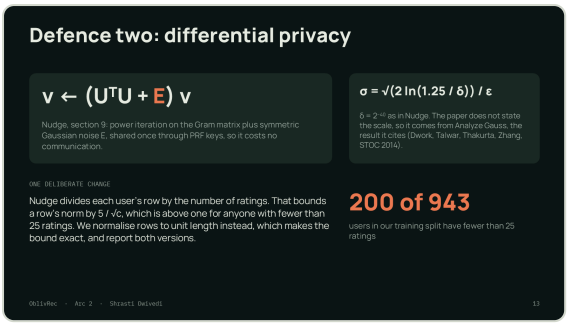


12 Defence one, decoys

1:15

On screen: **F6.** Attack strength against the number of decoys, with the popularity line drawn across.

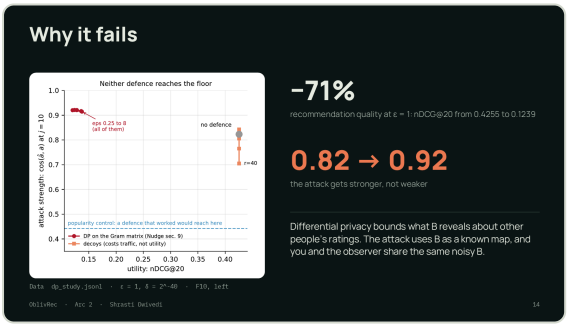
So can we avoid private retrieval with something cheaper? The obvious idea is decoys. Fetch your ten real items plus some extras to confuse the observer, drawn from popular items so that they look plausible. We measured it. Even with forty decoys, five times the real traffic, the attack only falls from 0.84 to 0.71. The popularity line sits at 0.44, and that line is where a defence has to land to have actually worked, because below it the observer has learned nothing about you that is not already public. Decoys do not get there. The reason is that averaging is robust. Adding popular items pulls the guess toward what is popular, which the control already knew.



13 Defence two, Nudge's own differential privacy 1:30

On screen: the mechanism. Power iteration runs on U transpose U plus a noise matrix E, the noise scale comes from Analyze Gauss, and rows are normalised by length because 200 of our 943 users have fewer than 25 ratings.

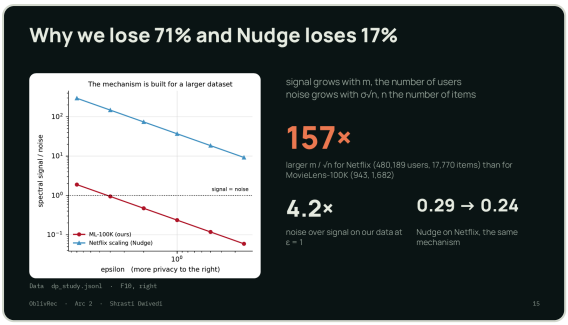
The second, more principled idea comes from Nudge itself. Its section nine adds differential privacy. We read that section before writing any code, and it was not what we expected. Nudge does not add noise to B. It adds Gaussian noise to the Gram matrix, U transpose U, through a noise term inside every step of power iteration. That costs no communication, which is elegant. The paper does not state the noise scale, so we took it from the result by Dwork and colleagues that it cites, known as Analyze Gauss. We also changed one detail on purpose. Nudge normalises each user's ratings by how many they gave, but that does not keep every user's contribution bounded. Two hundred of our 943 users have fewer than twenty-five ratings, which is exactly where the bound fails. So we normalised by length instead, which makes the guarantee exact, and we report both versions.



14 Why it fails 1:00

On screen: **F10, left panel.** Attack strength against recommendation quality, with the differential privacy points, the decoy points, and the popularity line, beside the two numbers that move the wrong way.

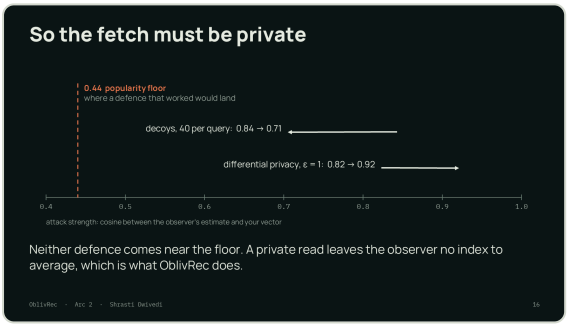
Both results go the wrong way. At epsilon equal to one, recommendation quality falls by 71 percent. And the attack does not get weaker. It gets stronger, from 0.82 to 0.92. That sounds like a paradox, but it is not. Differential privacy here protects the training data, meaning what B reveals about other people's ratings. Our attack asks a different question. It recovers your own vector, using B as a known map. You and the observer both use the same published, noisy B, so the geometry of the attack is untouched. What the noise does destroy is the shared popularity structure, and that structure was helping the control, not the attack.



15 Why we lose 71 percent and Nudge loses 17 1:00

On screen: **F10, right panel.** Signal-to-noise ratio against epsilon, for our dataset and scaled to Netflix.

There is one more thing worth explaining. Nudge reports that the same mechanism costs them only 17 percent on Netflix, while it costs us 71. Both numbers are right. The noise this mechanism adds grows with the square root of the number of items, while the useful signal grows with the number of users. Netflix has about half a million users and our dataset has under a thousand, which puts Netflix 157 times further into the safe region. We measured the ratio directly, and on our data the noise is about four times larger than the signal. So this mechanism is built for a much larger dataset than ours. That is a statement about scale, and not a disagreement with the paper.



16 So the fetch must be private 0:30

On screen: one scale of attack strength. Decoys move the attack from 0.84 to 0.71, differential privacy moves it from 0.82 to 0.92, and the popularity floor sits far away at 0.44.

So we measured two cheaper defences, and neither one reaches the line that would mean the observer learned nothing. That is the case for making the fetch itself private, which is what our system does. What does that cost?

[TRANSITION] Mainak.

Questions a viva will ask about Arc 2

Why is popularity the right control, and not a random guess? A random guess sits at zero and flatters any attack. Popular items top nearly everyone's list, so an estimator can score well having learned only what is public. Running the identical estimator on the most popular items isolates what is learned about this particular user. Evidence: the header of `model/attack.py`, figure F1.

Does the attack need anyone's ratings? No. It needs only B, which is public, and the indices of the items fetched. It never sees a rating.

Is the differential privacy result a flaw in Nudge? No. Nudge's differential privacy protects the training data and is sized for a dataset the scale of Netflix. Our measurement shows that it does not defend against this attack, and that it does not suit a dataset our size. The argument about users against the square root of items reconciles our 71 percent with their 17. Evidence: `model/dp_study.py`, figure F10, section 4 of `docs/threat-model.md`.

Why normalise by length rather than by count? Normalising by count bounds each user's contribution by five over the square root of their number of ratings, which is more than one for anyone with fewer than

twenty-five ratings, and 200 of our users are in that range. Normalising by length makes the bound exactly one, which is what the privacy guarantee needs.

ARC 3 · What it costs, and where it breaks

Mainak Sarkar · target 10:35

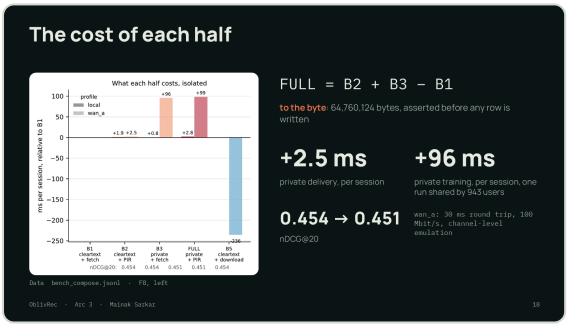


17 A distributed point function we can trust

0:45

On screen: "7.16 × 10⁹ checks. Every one passed.", beside the two-line invariant the tests assert at every node of the tree.

Thank you, Shrasti. Everything Shravan and Shrasti described rests on one primitive, the distributed point function, and we wrote it ourselves from the papers by Boyle, Gilboa and Ishai rather than using a library. So before any result, here is why you can trust it. We tested it exhaustively. For every possible secret index and every possible input, up to sixteen bits of domain, we compared its output against the true point function. That is 7.16 billion checks, and every one of them passed.

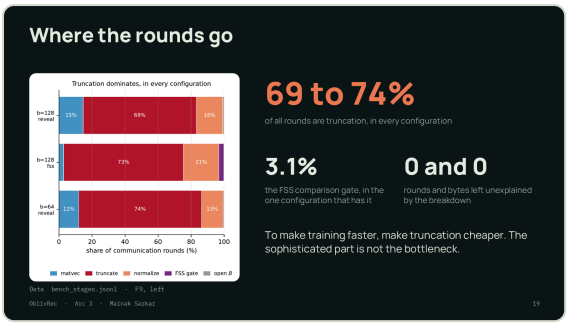


18 The cost of each half

1:30

On screen: **F8, left panel.** For each configuration, what it costs on top of the fully cleartext system.

Now, what does privacy cost? This figure isolates each half. First everything in the clear, then private delivery alone, then private training alone, then the full system with both. The first result is a clean one. The cost of the full system is exactly the cost of private delivery plus the cost of private training, to the byte. Composing the two halves adds nothing, because training traffic flows between the servers while delivery traffic flows between you and the servers, and the two never interact. We check that equality inside the benchmark before any number is written down. On a simulated wide-area link with thirty milliseconds of round-trip time, private delivery adds about two and a half milliseconds per session. Private training adds about ninety-six, once one training run is spread across the 943 users it serves. And recommendation quality barely moves, from 0.454 to 0.451.

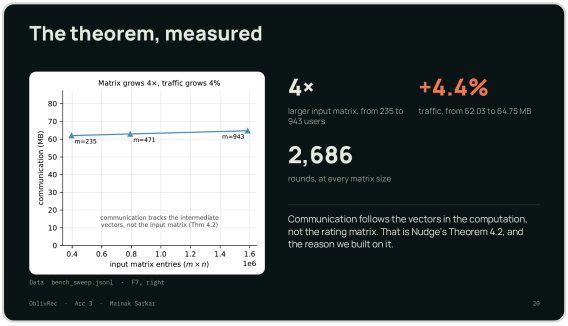


19 Where the rounds go

1:15

On screen: **F9, left panel.** The share of communication rounds taken by each operation, for three configurations.

Where does the time go? We attributed every communication round in training to the operation that caused it, and the attribution adds up with nothing left over: zero rounds and zero bytes unexplained, in every configuration we ran. The answer is not what we expected. Truncation, the operation that keeps fixed-point numbers in range, takes between 69 and 74 percent of all rounds. The comparison gate, the most sophisticated cryptography in the whole system, takes about 3 percent in the one configuration that uses it. So if you want this system to be faster, you make truncation cheaper. The sophisticated part is not the bottleneck.



20 The theorem, measured

0:45

On screen: **F7, right panel.** Traffic against input matrix size.

This one is short, and it is the reason we built on Nudge in the first place. Nudge proves that communication depends on the size of the vectors in the computation, and not on the size of the rating matrix. We measured it. We made the input matrix four times larger, and traffic grew by only 4.4 percent, while the number of rounds did not change at all.

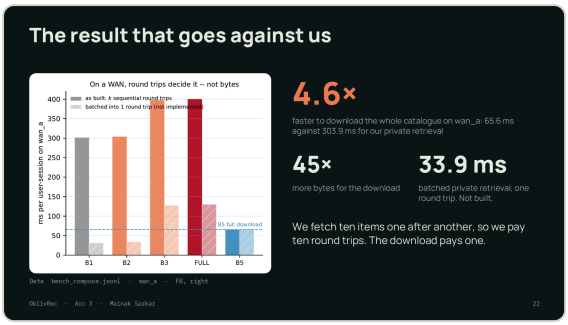


21 What measuring quality found

1:00

On screen: a bar chart. The first run, with the gate range too narrow: 0.3428 against 0.4513. After the fix: 0.4493 against 0.4513. Beside it, the rounds and bytes, identical both times, and a note that the first run's values are recorded in commit `d2138ad` and not in the committed data.

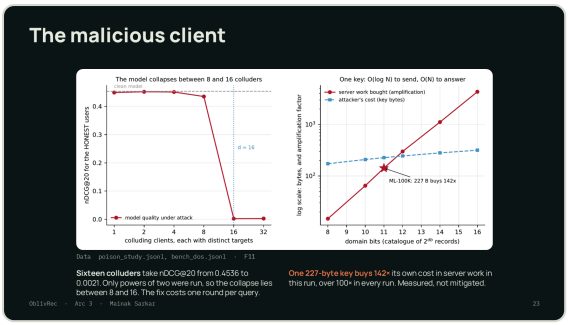
Now a story about why we measure quality and not just cost. The first time we measured the version of training that reveals nothing, it looked as if privacy cost us almost eleven points of recommendation quality. That would have been a striking headline, and it was wrong. A comparison gate had been given a range too narrow for the values it actually sees, and values outside that range were silently clamped. The number of rounds and bytes was identical either way, so a benchmark that measures only cost could never have caught it. Once fixed, the real cost is a few thousandths. We then confirmed it against a control run that changed one thing at a time.



22 The result that goes against us 1:00

On screen: **F8, right panel.** Time per session on the wide-area link, for our retrieval as built, for batched retrieval, and for downloading the whole catalogue.

Here is a result that goes against us, and we think you should hear it from us. On a wide-area link, simply downloading the entire catalogue is about four and a half times faster than our private retrieval, even though it sends about forty-five times more data. The reason is round trips. We fetch ten items one after another, so we pay ten round trips, and the full download pays one. On a slow network, round trips dominate. The fix is straightforward. You know all ten items at once, so you could ask for them in a single round trip, which would make us about twice as fast as the download. We have not built that, and our figures say so.

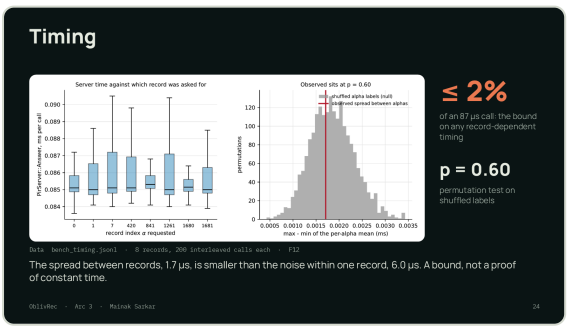


23 The malicious client

2:05

On screen: **F11.** Left, recommendation quality against the number of colluding clients, tested at 1, 2, 4, 8, 16 and 32, with a line at sixteen. Right, server work bought per byte of key, against catalogue size.

Now, what if a user is malicious? We found a real hole in our own design here. Remember the loop that collects training data from fetches. An honest client always asks for exactly one copy of an item, but nothing enforced that. A malicious client could ask for an item with a weight of one million, still get its film back, and cast a million-strong vote in the next round of training, with every check we had passing. So we measured what that buys. One attacker, even at a million, moves quality by only about one percent, because power iteration renormalises at every step, so extra weight only changes a direction. Eight attackers working together, each pushing different items, still cost only four percent. Sixteen destroy the model, and quality falls to almost zero. We tested powers of two, so the collapse happens somewhere between eight and sixteen. Our model has sixteen directions, which fits the idea that each attacker captures one, but we have not tested the numbers in between. The fix costs one round per query. The two servers add up their shares of the key's output and compare the totals, which reveals only the weight and never the item. It closes weight inflation. It does not stop a forged key that spreads the weight across items, and closing that needs an audit we have not built. The right panel shows a second exposure. One 227-byte key buys 142 times its own cost in server work in the run on screen, and more than a hundred times in every run we made, because the server has to touch every item to stay private. That one we measured but have not mitigated.

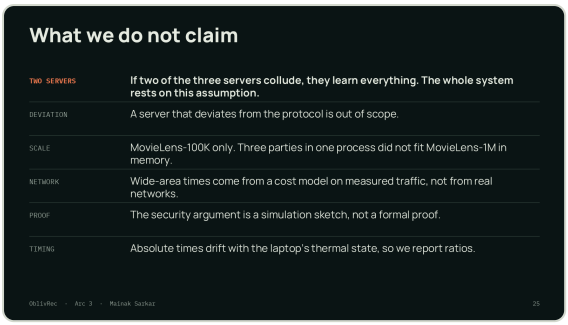


24 Timing

0:30

On screen: **F12.** Server time per call for eight different items, and the permutation test.

Finally, timing. Could an observer tell which item you fetched from how long the server takes? We timed the server's answer for eight items spread across the whole catalogue. The differences between items are smaller than the noise within a single item, so any leak is below 2 percent of a call. That is a bound, and not a proof of constant time.

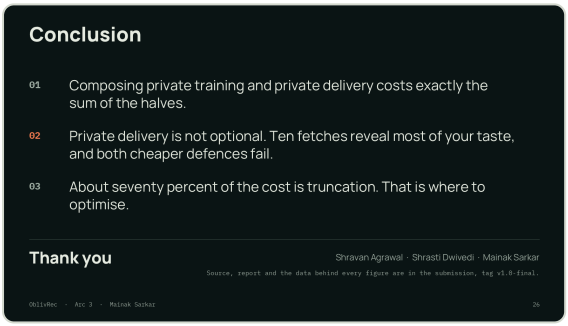


25 What we do not claim

1:00

On screen: the list below, one line each.

Let us be plain about what we do not claim. Our system is secure against one server that follows the protocol and tries to learn from what it sees. If two of the three servers collude, the shares reconstruct and there is no protection at all, and that is the assumption everything rests on. A server that deviates from the protocol is out of scope. We ran on MovieLens-100K and not the million-rating version, because simulating all three parties in one process did not fit comfortably in our machine's memory. Our wide-area numbers come from a cost model applied to measured traffic, not from real network emulation. Our security argument is a sketch and not a formal proof. And our timings vary between runs, so we report ratios rather than absolute times.



26 Conclusion

0:45

On screen: three lines. "Composing costs exactly the sum of the halves." "Private delivery is not optional." "About seventy percent of the cost is truncation."

To sum up. We built both halves of a private recommender and composed them, which neither paper had done. Composing them costs exactly the sum of the halves and nothing more. Private delivery is not optional, because ten observed fetches reveal most of your taste and both cheaper defences fail. And about seventy percent of the cost lies in one operation, truncation, which is where future work should go. Every figure you have seen is regenerated from committed data by one command. Thank you.

Questions a viva will ask about Arc 3

How do you know the cost breakdown is complete? Rounds and bytes are a closed form built from the protocol's own accounting, and it is checked against a real run. The residual is zero rounds and zero bytes in all three configurations. A second, independent check through a wrapper on the normaliser agrees as well. Evidence: `bench/bench_stages.cpp`, figure F9.

Why is your retrieval slower than downloading everything on a wide-area link? Delivery issues its ten fetches as ten sequential round trips, and the full download pays one. Batching the ten into one round trip would take 33.9 ms against the download's 65.6 ms. It is not implemented, and the figure labels it as such. Evidence: `bench/bench_compose.cpp`, figure F8.

Is the collapse exactly at sixteen attackers? We cannot say that. We tested 1, 2, 4, 8, 16 and 32 colluders. Eight cost 4.1 percent and sixteen cost 99.5 percent, so the collapse is somewhere between eight and sixteen. The model has sixteen directions, and one attacker capturing each direction would explain a collapse at sixteen, but the numbers in between are untested. Evidence: `model/poison_study.py`, figure F11.

Does the weight check leak anything? It opens the difference of the two servers' sums, which equals the weight exactly, whatever the item. The weight is supposed to be the public constant one, so opening it reveals nothing about which item was fetched. Evidence: `src/pir/harvest.cpp`, `tests/test_harvest.cpp`.

Why write your own distributed point function? Project rule: the cryptography is written from the papers, and third-party code appears only as a benchmark baseline. The exhaustive test is what makes a hand-written one trustworthy. Evidence: `tests/test_dpf_exhaustive.cpp`.

Every number in this script, and where it comes from

Each value is taken from the newest git sha in its results file, the same rule `bench/scripts/make_figures.py` uses. A number without its setting is misleading, so the setting is given as well.

SLIDE	CLAIM	VALUE	SETTING	SOURCE
6	DPF key size	227 B	2,048-entry domain	`tests/test_dpf_serialize.cpp`, F2
8	Demo output	10 titles, byte-exact, ranking matches on 10 of 10	user 42, no-leak model, $\ell = 10$	`src/apps/demo.cpp`
10	Attack, 1 fetch	cos 0.55	ML-100K, d = 16	`leakage_attack.jsonl`, F1
10	Attack, 10 fetches	cos 0.84, 56% of next 20	ML-100K, d = 16	`leakage_attack.jsonl`, F1
11	Popularity control	cos 0.44	10 fetches	`leakage_attack.jsonl`, F1
11	Margin over control	about 0.40 (0.38 to 0.42 for 2 to 50 fetches)	0.28 at 1 fetch	`leakage_attack.jsonl`
12	Decoys	0.84 to 0.71 at 40 decoys	5× traffic	`leakage_attack.jsonl`, F6
13	Users under 25 ratings	200 of 943	ML-100K u1 split	`model/dp_study.py` output
14	DP utility	nDCG@20 0.4255 to 0.1239, −71%	$\varepsilon = 1$, $\delta = 2^{-40}$, length-normalised	`dp_study.jsonl`, F10
14	DP attack	cos 0.82 to 0.92	$\varepsilon = 1$	`dp_study.jsonl`, F10
15	Nudge's own cost	0.29 to 0.24, −17%	Netflix, $\varepsilon = 1$	Nudge section 9

SLIDE	CLAIM	VALUE	SETTING	SOURCE
15	Scale ratio	157×	m / √n, Netflix against ML-100K	`model/dp_study.py` header
15	Noise against signal	about 4.2×	ε = 1	`dp_study.jsonl`
17	DPF correctness	7.16×10^9 checks	every index against every input, domains up to 16 bits at b = 64 and 15 at b = 128	`tests/test_dpf_exhaustive.cpp`
18	Additivity	FULL = B2 + B3 − B1 = 64,760,124 B	ML-100K, d = 16, ℓ = 10, b = 64	`bench_compose.jsonl`, F8
18	Private delivery	+2.48 ms per session	wan_a, 30 ms RTT, 100 Mbit/s	`bench_compose.jsonl`, F8
18	Private training	+96.3 ms per session	wan_a, spread over 943 users	`bench_compose.jsonl`, F8
18	Quality	0.454 to 0.451	ℓ = 10, revealing normaliser	`train_quality.jsonl`
19	Truncation share	68.6% to 74.4% of rounds	three configurations	`bench_stages.jsonl`, F9
19	Gate share	3.1% of rounds	b = 128, no-leak normaliser only	`bench_stages.jsonl`, F9
19	Residual	0 rounds, 0 bytes	all three configurations	`bench_stages.jsonl`
20	Theorem 4.2	4× matrix, +4.4% traffic, rounds unchanged at 2,686	m from 235 to 943	`bench_sweep.jsonl`, F7
21	Gate range bug, before the fix	0.108 apparent (0.3428 against 0.4513)	ℓ = 10, b = 128	**Not in committed results.** Recorded in commit `d2138ad` and the Decisions Log. Reproducible by setting the gate range back to `[t−8, t+8]` in `src/apps/train.cpp`
21	No-leak cost, after the fix	0.0018 to 0.0041 of nDCG@20	ℓ ≥ 2, against either revealing control	`train_quality.jsonl`
22	Full download against PIR	4.6× faster, 44.6× the bytes	wan_a, k = 10	`bench_compose.jsonl`, F8
22	Batched PIR	33.9 ms against 65.6 ms	wan_a, not implemented	`bench_compose.jsonl`, F8
23	One attacker	−0.7% to −1.0%	weight 10^6, two runs with different targets	`poison_study.jsonl`, F11
23	Eight colluders	−4.1%	distinct targets	`poison_study.jsonl`, F11
23	Sixteen colluders	nDCG@20 0.4536 to 0.0021	d = 16. Only 1, 2, 4, 8, 16 and 32 were tested, so the collapse lies between 8 and 16	`poison_study.jsonl`, F11

SLIDE	CLAIM	VALUE	SETTING	SOURCE
23	Weight check cost	1 round, 2 ring elements	per query	`tests/test_harvest.cpp`
23	DoS amplification	142× in the run the figure shows, 121× in an earlier committed run	2,048-entry domain	`bench_dos.jsonl`, F11
24	Timing bound	below 2.0% of an 87 µs call, p = 0.60	8 items, 200 calls each	`bench_timing.jsonl`, F12
26	Truncation, rounded	about seventy percent	as slide 19	`bench_stages.jsonl`

Before recording

- Regenerate the figures with `mingw32-make figures` from the frozen tag, and take every figure from `report/figures/`, never from an older export.
- Run the slide 8 demo command once and record the real terminal output. Do not re-type it.
- Rehearse with a timer. The video must come in **under** 30 minutes.
- Each speaker rehearses the viva questions at the end of their arc.
- Upload to YouTube as **unlisted**, and check the link in a private browser window before submitting it.
- Before recording, search this file and the deck for em dashes and semicolons. The project's writing rule covers the narration script as well as the report.